ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 2

INTRODUCTION TO SYSTEMS DEVELOPMENT

PRACTICE EXAM · MARKING GUIDELINE

MARKS: 100

Marking guideline for the Academic Central NC(V) Level 2 Introduction to Systems Development practice exam.

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 2. It is not a Department of Higher Education and Training marking guideline.

How to mark: each ✓ is one mark. Accept answers in the student's own words if the meaning is correct. Accept any correct pseudocode style or flowchart that gives the right result; do not penalise small differences in keywords.

SECTION A

QUESTION 1

1.1 1.1.1 D ✓ 1.1.2 A ✓ 1.1.3 C ✓ 1.1.4 C ✓ 1.1.5 A ✓ 1.1.6 D ✓ 1.1.7 B ✓ 1.1.8 D ✓ 1.1.9 B ✓ 1.1.10 C ✓ (10)

1.2 1.2.1 False ✓ 1.2.2 True ✓ 1.2.3 True ✓ 1.2.4 False ✓ 1.2.5 True ✓ 1.2.6 True ✓ 1.2.7 False ✓ 1.2.8 False ✓ 1.2.9 True ✓ 1.2.10 True ✓ (10)

1.3 1.3.1 F ✓ 1.3.2 D ✓ 1.3.3 J ✓ 1.3.4 G ✓ 1.3.5 E ✓ 1.3.6 H ✓ 1.3.7 B ✓ 1.3.8 C ✓ 1.3.9 A ✓ 1.3.10 I ✓ (10)

TOTAL SECTION A: [30]

SECTION B

QUESTION 2

2.1 Planning (preliminary investigation) ✓; analysis ✓; design ✓; implementation (development, testing and conversion) ✓; maintenance (review / support) ✓ – in this order (5)

2.2 Study the current paper system ✓; gather facts from the owners, drivers and marshals (interviews, questionnaires, observation) ✓; identify problems and the requirements of the new system ✓. (3)

2.3 Any TWO with an advantage: interview ✓ – can ask follow-up questions / get detailed answers ✓; questionnaire ✓ – reaches many people quickly and cheaply / anonymous ✓; observation – see how work is really done; document review – shows actual data used (4)

2.4 Direct changeover: the old system stops and the new system starts on the same day ✓ – cheap and fast but risky ✓. Parallel running: old and new systems run together for a period ✓ – safe, results can be compared, but double work and costly ✓. Recommendation with reason, e.g. parallel running, because daily takings (money) must not be lost if the new system fails ✓ (accept direct with a sensible reason) (5)

2.5 Any THREE: to fix errors (bugs) found after implementation ✓; to adapt to new requirements, laws or tax rates ✓; to improve performance or add features ✓; to keep up with new hardware or operating systems; security updates (3)

[20]

QUESTION 3

3.1 Example (✓ for each part):

```
INPUT amount                          ✓
IF amount > 50 THEN                   ✓
   discount ← amount * 0.1            ✓
ELSE
   discount ← 0                       ✓
ENDIF
toPay ← amount - discount             ✓
OUTPUT "Amount to pay: R", toPay      ✓
```

(6)

3.2 Oval Start ✓ → parallelogram Input amount → diamond "amount > 50?" ✓ with Yes/No branches labelled ✓ → rectangle discount = amount × 0,1 (Yes) and discount = 0 (No) ✓ → rectangle toPay = amount − discount ✓ → parallelogram Output toPay → oval Stop ✓ (correct symbols and flow lines) (6)

3.3 Input: amount ✓; Processing: test if amount > 50, calculate discount and amount to pay ✓; Output: amount to pay ✓ (3)

[15]

QUESTION 4

4.1 Sensible name (camelCase or similar, no spaces) and type, 1 + 1 each: (a) numStudents ✓ – integer ✓ (b) breadPrice ✓ – real / float / currency ✓ (c) hasPaid ✓ – Boolean ✓ (6)

4.2 (a) 7 + 6 = 13 ✓ (b) 10 / 2 = 5 ✓ (c) 1 ✓ (d) 2 ✓ (4)

4.3 4.3.1 Trace table (✓ per correct row, ✓✓ for correct final values):

i	mark	count	total
–	–	0	0
1	60	1	60
2	35	1	95
3	80	2	175
4	45	2	220

(6)

4.3.2 55 ✓ and 2 ✓ (average 220 / 4 = 55; count 2) (2)

4.3.3 The number of marks of 50 or more (number of students who passed) ✓ (1)

4.3.4 Example:

```
i ← 1                                  ✓
WHILE i <= 4                           ✓
   INPUT mark
   IF mark >= 50 THEN count ← count + 1
   total ← total + mark                ✓ (body unchanged)
   i ← i + 1                           ✓
ENDWHILE
```

(4)

4.3.5 Selection: lines 5–7 ✓; iteration: lines 3–9 ✓ (2)

[25]

QUESTION 5

5.1 Normal: any age from 16 to 34, e.g. 20 ✓; boundary: 15 or 35 ✓; erroneous: e.g. 40, –3, 12 or "abc" ✓ (3)

5.2 Syntax error: breaks the rules (grammar) of the language, so the program will not run / compile ✓, e.g. a misspelt keyword such as PRNT or a missing bracket ✓. Logic error: the program runs but gives wrong results ✓, e.g. using + instead of − or > instead of >= ✓. (4)

5.3 Any TWO: user documentation (user manual / help files) ✓ – used by end users to learn how to use the system ✓; technical documentation (program code comments, data dictionary, system design) – used by programmers / maintenance staff ✓ (1 for each type, 1 for users; max 3) (3)

[10]

TOTAL SECTION B: [70]

TOTAL: 100

END OF MARKING GUIDELINE